

Tool_Box v1.0

External Module Creation Guide

A complete tutorial for building, loading, customizing, testing, and troubleshooting Tool_Box external modules.

Purpose of this guide

This PDF is focused specifically on the external-module system introduced in Tool_Box. It is intended to stand alone from the main README.

AI-assisted / vibe-coded project

Tool_Box was created and developed with substantial AI assistance. Review and test the code in a controlled environment before relying on it.

Version 1.0

Prepared as standalone module documentation

Contents

- 1. Important disclaimer and authorized-use notice 3
- 2. What an external module is 3
- 3. Full tutorial: create your first module 4
- 4. Placeholders 5
- 5. Option groups and reserved placement placeholders 5
- 6. Command parsing and quoting rules 6
- 7. Save, load, and test the module 7
- 8. Using a loaded external module 8
- 9. Password handling 9
- 10. Complete example: URL-based curl module 9
- 11. Complete example: Nmap with positioned options 10
- 12. Updating, reloading, and persistence 10
- 13. Supported categories 11
- 14. Troubleshooting 12
- 15. Security, privacy, and known limitations 13
- 16. Writing good modules 13
- 17. Final checklist before pressing R 14

1. Important disclaimer and authorized-use notice

AI-assisted / vibe-coded project

Tool_Box was created and developed with substantial AI assistance ("vibe coded"). AI-generated or AI-modified code can contain mistakes, incomplete validation, unexpected behavior, or assumptions that have not yet been identified. Review the code and test it before using it on systems that matter.

Tool_Box is provided as-is, without warranties or guarantees of any kind. Use of the project is entirely at your own risk. The author is not responsible for misuse, direct or indirect damage, data loss, service interruption, system instability, security incidents, legal issues, financial loss, or other consequences resulting from its use.

You are responsible for ensuring that you have **explicit authorization** to scan, enumerate, test, access, or otherwise interact with every system, network, service, account, or data target used through Tool_Box.

- Use Tool_Box only on systems you own, administer, or have clear permission to test.
- Training platforms, CTF environments, and isolated lab machines are appropriate when their rules allow the activity.
- Third-party programs launched by Tool_Box have their own licenses, defaults, risks, and documentation.
- Some commands can generate heavy traffic, alter system state, require elevated privileges, or expose sensitive data in third-party output.
- This guide is not legal advice. If authorization is unclear, do not run the command until you have confirmed permission.

Recommended safety practice

Use Dry-Run for new modules, review the full Command Preview, keep backups of important data, and test new modules in an isolated lab first.

2. What an external module is

An external module is a plain **.txt** definition file placed inside the **Toolboxmodules** folder. It adds a command to a Tool_Box menu without requiring you to edit the main Bash script.

A module tells Tool_Box four basic things: what the module is called, what it does, where it should appear, and what command template it should build. It can also define optional toggles and placeholders.

The module manager is located at:

```
System & Configuration -> Settings -> External Modules
```

From there you can list discovered modules, load one, load all, unload one, unload all, or rescan/reload selected modules. Loading a definition does **not** execute its command.

A typical project layout looks like:

```
Tool_Box/  
|-- ToolBox.sh  
|-- README.md  
`-- Toolboxmodules/  
    |-- Basic_nmap.txt  
    `-- Basic_curl.txt
```

3. Full tutorial: create your first module

Step 1 - Decide what the module should do

Start with one specific task. A good first module usually wraps one program and one general action, such as a basic Nmap scan, a curl request, a DNS query, a WHOIS lookup, or another repeatable command used in your authorized lab.

Keep the first version simple. Add optional toggles after the base command works.

Step 2 - Create the .txt file

Create the file directly inside **Toolboxmodules**:

```
nano Toolboxmodules/My_First_Module.txt
```

- The filename must end in .txt.
- Nested module folders are not scanned.
- Symbolic links are ignored.
- UTF-8 text, Windows line endings, Linux line endings, and an optional UTF-8 BOM are accepted.
- Blank lines are allowed.
- Whole-line comments beginning with # are allowed.

Step 3 - Add the four required fields

```
Name: Example_Name  
Description: A short explanation of what this module does.  
Category: Reconnaissance  
Command: example-program <<IP>>
```

Name: the menu name. Keep it short and descriptive. Duplicate module names are rejected case-insensitively, and names that conflict with registered built-in tools are rejected.

Description: the explanation shown on the module page. Describe the purpose, not just the command name.

Category: controls which Tool_Box menu displays the module. Category names are case-insensitive, but must match a supported category or accepted alias.

Common spelling mistake

Use **Category:** - not **Catagory:**. The misspelled field is rejected.

Command: the base command template. The first token must be a real executable name; the executable itself cannot be a placeholder.

4. Placeholders

Placeholders let modules reuse values already configured in Tool_Box. Use the exact uppercase syntax `<<VARIABLE>>`.

Placeholder	Value supplied by Tool_Box
<code><<IP>></code>	Current global target
<code><<PORT>></code>	Current port selection, including lists or ranges
<code><<SUBNET>></code>	Current prefix length, such as 24
<code><<DOMAIN>></code>	Domain set under Set Target/Data
<code><<URL>></code>	URL set under Set Target/Data
<code><<WORDLIST>></code>	Default wordlist configured in Settings
<code><<USERNAME>></code>	Username set under Set Target/Data
<code><<PASSWORD>></code>	Password set under Set Target/Data
<code><<WORKSPACE>></code>	Active workspace name

Example using IP and subnet:

```
Command: nmap <<IP>>/<<SUBNET>>
```

If IP is **192.168.56.0** and subnet is **24**, the resolved target becomes **192.168.56.0/24**.

If a required placeholder is empty when you press **R**, Tool_Box prompts for it. Blank input cancels the run. Custom placeholder names can also be prompted at run time, but Tool_Box does not know what kind of data an unknown placeholder is supposed to contain.

5. Option groups and reserved placement placeholders

Tool_Box v1.0 supports up to three independent option groups. Each group contains normal ON/OFF option entries, while a reserved placeholder in **Command:** controls exactly where the enabled arguments are inserted.

The three section/marker pairs are:

Section	Reserved command marker	Behavior
options:	<code><<OPTIONS>></code>	Primary option group. If the marker is omitted, enabled options keep legacy append-at-end behavior.
options2:	<code><<OPTIONS2>></code>	Second positioned group. The matching marker is required.
options3:	<code><<OPTIONS3>></code>	Third positioned group. The matching marker is required.

Example with all three groups:

```

Name: Example
Description: Example with three positioned option groups.
Category: Reconnaissance
Command: program <<OPTIONS>> subcommand <<OPTIONS2>> <<IP>> <<OPTIONS3>>

options:
  verbose: -v
  quiet: -q

options2:
  ports: -p <<PORT>>
  service detection: -sv

options3:
  output file: -oN <<OUTPUT_FILE>>

```

If **verbose**, **ports**, and **output file** are enabled, Tool_Box inserts those argument fragments at their matching reserved markers instead of simply placing everything at the end.

```
program -v subcommand -p 80,443 192.168.56.10 -oN scan.txt
```

Each option entry still uses **label: arguments**. Options are displayed under separate **Options**, **Options 2**, and **Options 3** headings, but the menu keeps one continuous numbering sequence.

Backward compatibility

Existing modules do not need to be rewritten. If a module has an **options:** section but does not use **<<OPTIONS>>**, enabled entries from that first group are appended to the end of the base command exactly as before.

Validation rules

- Option sections must appear after Name, Description, Category, and Command.
- Use the section order options:, then options2:, then options3:.
- **<<OPTIONS>>**, **<<OPTIONS2>>**, and **<<OPTIONS3>>** must be standalone tokens in Command: and may appear only once each.
- options2: requires **<<OPTIONS2>>** in Command:.
- options3: requires **<<OPTIONS3>>** in Command:.
- A reserved marker without its matching section is rejected.
- Unsupported reserved forms such as **<<OPTIONS4>>** are rejected.
- Reserved OPTIONS markers are structural and do not appear in Custom Placeholder Values.
- Normal placeholders inside option entries, such as **<<PORT>>**, **<<USER_AGENT>>**, or **<<OUTPUT_FILE>>**, still work normally.

Option toggles reset when you leave and reopen the module. A normal placeholder used only by a disabled option is not required when the command runs, but it is still available in Custom Placeholder Values so you can prepare the value before enabling that option.

6. Command parsing and quoting rules

External module definitions are treated as **data**, not sourced as Bash code. The parser supports a program plus arguments, single quotes, double quotes, backslash escaping, and placeholders inside arguments.

```
Command: curl -H "User-Agent: Tool_Box" <<URL>>
```

Quote arguments that contain spaces.

The external-module format intentionally does **not** provide automatic shell evaluation. It does not automatically support:

- Pipes such as |
- Redirection such as > or <
- Command substitution such as \$(command)
- Backtick command substitution
- Automatic glob expansion
- Environment-variable expansion
- Arbitrary shell expressions

Why this restriction exists

The parser deliberately avoids turning a text definition into arbitrary shell code. For a complex multi-command workflow, write and review a separate script instead of hiding shell logic inside the module definition.

7. Save, load, and test the module

A complete first module can look like:

```
# Simple Nmap example for Tool_Box

Name: Basic_nmap
Description: Basic Nmap scan using the current Tool_Box target.
Category: Reconnaissance
Command: nmap <<OPTIONS>> <<IP>>
options:
  port: -p <<PORT>>
  version detection: -sV
  default scripts: -sC
```

Save it as **Toolboxmodules/Basic_nmap.txt**.

Start Tool_Box and navigate to **System & Configuration -> Settings -> External Modules**. List discovered modules to confirm the file is visible, then choose Load one or Load all.

A loaded module appears in its category using an external selector such as:

```
E1) Basic_nmap [external]
```

External numbers are shared across categories, so gaps such as E1, E4, and E7 are normal.

Test with Dry-Run first

Dry-Run is the preferred first test for a new module. Check placeholder substitution, URL formatting, port syntax, quoting, enabled options, custom arguments, and the final target before live execution.

8. Using a loaded external module

A v1.0 external module page is organized into clear sections. Modules that define more than one option group show each group separately while keeping one continuous number sequence:

```

Description:
  Example command using three option groups.

----- Options -----
1) Verbose      : OFF
2) Quiet       : OFF

----- Options 2 -----
3) Ports       : OFF
4) Service Detect : OFF

----- Options 3 -----
5) Output File  : OFF

----- Customization -----
C) Custom Placeholder Values
A) Custom Arguments
X) Clear Customizations

----- Command Preview -----
Command: program subcommand 192.168.56.10

----- Actions -----
R) Run

----- Documentation -----
D) Module Help

[0] Back    [M] Main Menu    [T] Set Target/Data    [W] Workspace    [H] Help

```

The footer already contains **[0] Back**, so Back is not repeated inside the menu body.

Custom Placeholder Values

Press **C** to override normal placeholders for this module only. Reserved option-placement markers are not editable values and are intentionally omitted from this screen.

Resolution order:

```

module-local override
|
Tool_Box global/default value
|
prompt when R is selected

```


Module overrides do not change Set Target/Data, saved profiles, or the module .txt file. Reopening the module starts with a clean set of module-local customizations. Blank input while editing a placeholder clears that one override.

Custom Arguments

Press **A** to append temporary arguments after the fully resolved module command. Custom Arguments use the same non-evaluating tokenizer as module definitions. Quoted normal arguments are supported; shell pipelines, redirection, command substitution, and other shell-evaluation syntax are rejected.

Press **X** to clear both module-local placeholder overrides and Custom Arguments.

9. Password handling

The <<**PASSWORD**>> placeholder receives additional protection inside Tool_Box:

- Password input is hidden when Tool_Box requests it.
- Password values are redacted from Tool_Box command previews.
- Password values are redacted from Tool_Box command history.
- Password values are not written to the normal Tool_Box configuration.
- Password values are not exported into restarted/elevated Tool_Box sessions.

Important limitation

A third-party command can still display, log, transmit, or otherwise expose a password. Tool_Box cannot control what another program does with the arguments it receives. Do not store real credentials directly in module definition files.

10. Complete example: URL-based curl module

Create **Toolboxmodules/Basic_curl.txt** with:

```
# v1.0 example using all three positioned option groups

Name: Basic_curl
Description: HTTP/HTTPS request demonstrating OPTIONS, OPTIONS2, and OPTIONS3 placement.
Category: Web Enumeration
Command: curl <<OPTIONS>> <<OPTIONS2>> <<URL>> <<OPTIONS3>>

options:
  Follow redirects: -L
  Ignore TLS certificate errors: -k

options2:
  Headers only: -I
  User agent: -A <<USER_AGENT>>

options3:
  Output file: -o <<OUTPUT_FILE>>
```

Load the module and open it from **Web Enumeration**. The first group is placed before the second group, the URL stays exactly where it appears in Command:, and the third group is placed after the URL.

If the global URL is not the one you want:

- Press C.
- Select URL.
- Enter the temporary URL.
- Return to the module and confirm the new value in Command Preview.
- Enable any desired options from any group.
- If you enable User agent or Output file, provide those normal placeholder values when prompted or through Custom Placeholder Values.
- Use Dry-Run for a final check.
- Press R only when you are ready to run the command on an authorized target.

11. Complete example: Nmap with positioned options

```
# Example Nmap module

Name: Lab_nmap
Description: Nmap scan using the current Tool_Box target, subnet, and positioned scan options.
Category: Reconnaissance
Command: nmap <<OPTIONS>> <<IP>>/<<SUBNET>>
options:
  ports: -p <<PORT>>
  service detection: -sV
  default scripts: -sC
```

One possible resolved command is:

```
nmap -p 22,80,443 -sV 192.168.56.0/24
```

Because **<<OPTIONS>>** appears before the target, enabled Nmap options are inserted there. If you removed the marker but kept the options: section, Tool_Box would preserve legacy behavior and append the enabled options after the base command instead.

12. Updating, reloading, and persistence

After editing a module that is already selected:

- Save the .txt file.
- Open System & Configuration -> Settings -> External Modules.
- Choose Rescan / reload selected modules.
- Review any warnings.
- Reopen the external module and verify the Command Preview.

- Use Dry-Run before live execution.

If a selected module becomes invalid, it can remain selected but unavailable until repaired or unloaded. If you rename a file, load/select the new filename and unload the old one.

Selected module filenames are persisted in:

```
~/ .tool_box.modules.conf
```

This selection file is separate from normal Save Configuration. Selections are per Linux user, so a normal user and root can have different module selections. New files are discovered by rescan but are not automatically selected.

13. Supported categories

Use one of these supported category or subcategory names:

Target & Scanning

- Workspace / Project
- Scan Profiles

Reconnaissance & Enumeration (Recon & Enumeration)

- Reconnaissance
- Web Enumeration
- Network / DNS
- SMB / Windows
- Service Enumeration
- Vulnerability Assessment

Access & Exploitation

- Pass/Hash Cracking (Pass / Hash Cracking)
- Exploit

Analysis & Reporting

- Traffic Analysis
- Results / Reporting

Utilities & Resources

- Command-Line Utilities (Utilities)
- Network Information (Network Information / VPN Status)
- Text / Encoding Tools
- Useful Links
 - Shell / Session Tools
 - Data / Encoding / Analysis
 - Privilege References (Linux / Windows Privilege References)
 - Web Security
 - Recon / Vulnerabilities (Recon / Vulnerability References)
 - Payloads / Wordlists / Cheatsheets (Payloads / Wordlists / Cheat Sheets)

- Training

System & Configuration

- Settings
- Documentation

14. Troubleshooting

Module does not appear

- Confirm the file is directly inside Toolboxmodules beside the script.
- Confirm the filename ends in .txt and is not a symbolic link.
- Confirm the module is loaded/selected in External Modules.
- Confirm Category is supported and the module parsed without an error.
- Use Rescan / reload selected modules after edits.

Required field is missing

```
Name:  
Description:  
Category:  
Command:
```

Each of the four required fields must contain a value.

Unknown field

Check spelling. **Category:** is valid; **Catagory:** is not.

Executable unavailable

Tool_Box checks whether the executable at the start of the resolved command is available in PATH. Test it manually:

```
command -v PROGRAM_NAME  
  
# Example  
command -v nmap
```

Placeholder keeps prompting

The global value may be empty, the override may have been cleared, or the placeholder may be custom. Use **C -> Custom Placeholder Values** to prepare a module-local value.

Option is not being added

Confirm it appears under the intended options:, options2:, or options3: section; has a label and arguments separated by a colon; is toggled ON; and its arguments pass the tokenizer. options2: and options3: also require their matching reserved marker in Command:.

Reserved option marker is rejected

Use only <<OPTIONS>>, <<OPTIONS2>>, or <<OPTIONS3>>. Each must be a standalone Command token and may appear only once. <<OPTIONS4>> and embedded forms such as prefix<<OPTIONS>>suffix are rejected.

Pipes or redirection are rejected

That is intentional. External modules are not arbitrary shell scripts. Create a separate reviewed script for multi-command shell workflows.

Edited module still looks old

Use **External Modules -> Rescan / reload selected modules**, then reopen the module.

15. Security, privacy, and known limitations

Security practices

- Review Command Preview before every important run.
- Use Dry-Run when testing a new module or option.
- Do not put passwords, API tokens, private keys, or other secrets directly in module files.
- Remember that third-party tools can print secrets even when Tool_Box redacts its own history.
- Be cautious when running Tool_Box as root.
- Verify downloaded tools and repositories before installing them.
- Keep important data backed up and use isolated lab networks for scan testing.
- Review unknown module definitions before loading them.

Known limitations

- A module represents one executable command template rather than a full shell script.
- A module supports up to three positioned option groups: options:, options2:, and options3:.
- Pipelines, redirection, command substitution, environment-variable expansion, and glob expansion are intentionally unsupported.
- Nested module directories are not scanned and symbolic-link modules are ignored.
- Module-local customizations are temporary and reset when the module is reopened.
- Auto-Save uses Tool_Box's existing text-output pipeline; some fully interactive programs may not fit this module format.
- A module can validate structurally and still contain arguments that are wrong for the installed version of a third-party tool.
- Tool_Box cannot determine whether a target or action is authorized simply because a command appears in a menu.

16. Writing good modules

- Give the module a descriptive name.
- Explain its purpose in Description.
- Use the narrowest appropriate category.
- Keep the base command simple.
- Put optional behavior into clearly named toggles.
- Use placeholders instead of hard-coding environment-specific values where practical.
- Never hard-code real passwords or tokens.
- Add # comments for non-obvious behavior.
- Test with Dry-Run first.
- Confirm third-party syntax using the program's own documentation.
- Document software dependencies.
- Share only modules you have reviewed and tested in an authorized environment.

Reusable starter template

```
# -----  
# Tool_Box external module  
# Replace the example values below with your own module.  
# -----  
  
Name: My_Module  
Description: Describe what this module does and when you would use it.  
Category: Reconnaissance  
Command: program <<OPTIONS>> <<IP>> <<OPTIONS2>> <<OPTIONS3>>  
options:  
    first option: --example  
    port: -p <<PORT>>  
options2:  
    second-stage option: --mode <<MODE>>  
options3:  
    output file: -o <<OUTPUT_FILE>>
```

After creating it: save the file in Toolboxmodules, load it from External Modules, open the category you assigned, verify Command Preview, test with Dry-Run, and rescan/reload after corrections.

17. Final checklist before pressing R

- Confirm the target is exactly the one you intend.
- Confirm you are authorized to test it.
- Review the entire Command Preview.
- Check whether elevated privileges are required.
- Consider Dry-Run first.
- Understand what the third-party program is expected to do.
- Check output paths for sensitive or important data.
- Keep backups when a command could change system state.

Final reminder

Tool_Box is a convenience layer, not a substitute for understanding the commands being run. Use it responsibly and keep security testing inside environments where you have permission.

License note

This guide does not create or imply an open-source license. If the project repository does not contain a LICENSE file, do not assume a particular license applies. Choose a license deliberately before publishing the project for redistribution or accepting outside contributions.

Third-party tool and project names remain the property of their respective owners. Inclusion in Tool_Box does not imply affiliation, endorsement, sponsorship, or warranty by those maintainers.